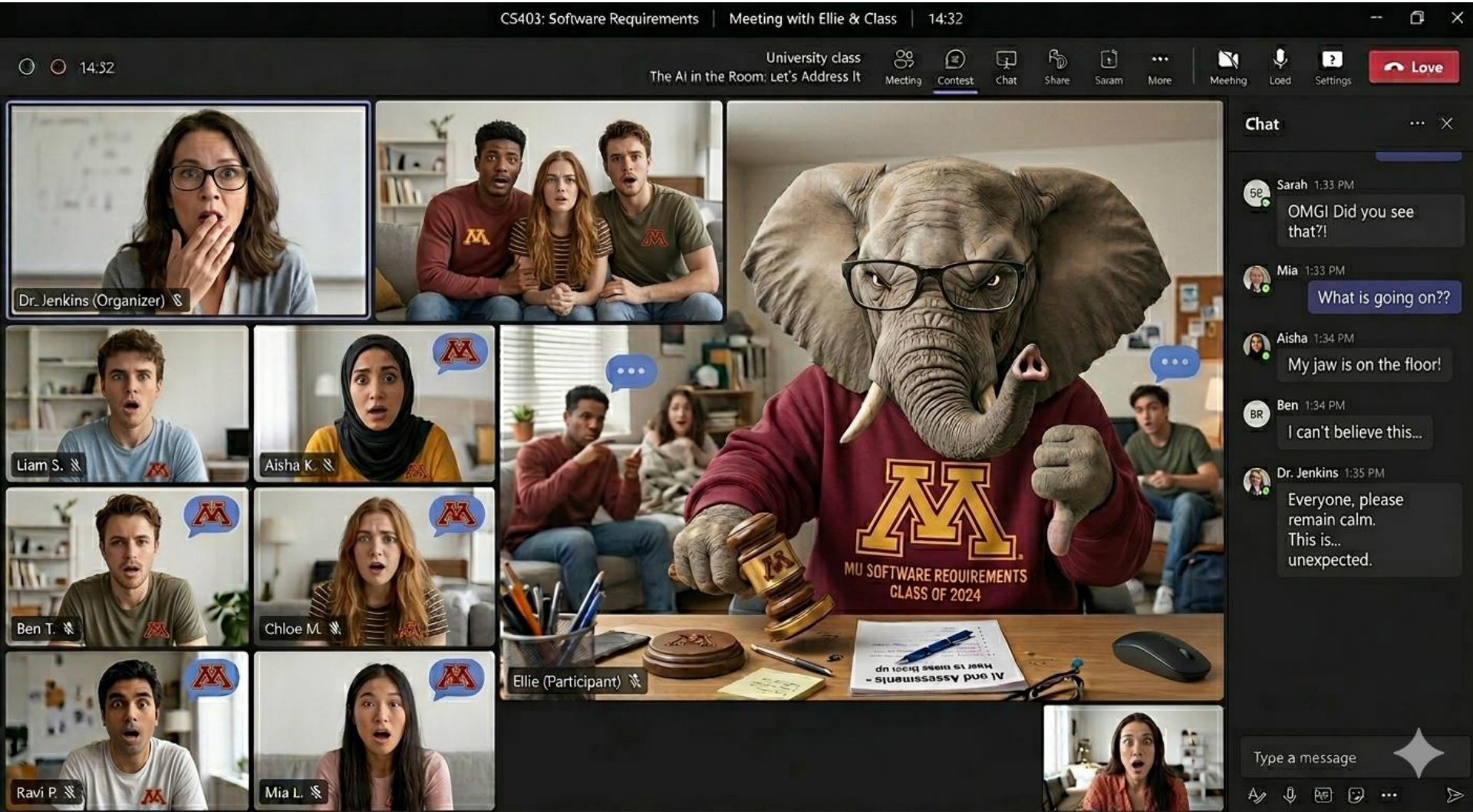
Agentic Engineering

There's an elephant in the (virtual) room we can't avoid talking about...



AI and software development

- Modern AI tools are having a substantial and rapidly evolving impact on software development
- This makes this topic simultaneously highly difficult to cover, but also very important
- This will therefore not be a perfect coverage of the topic (if that's even possible), but let's dive in anyways because we need to talk about it

AI is being ***widely*** adopted...

90% of Developers Now Use AI Coding Tools at Work

The debate about whether AI coding tools are mainstream is over. JetBrains ran its second AI Pulse survey in January 2026 — a globally representative study of over 10,000 professional developers, localized into eight languages — and the headline number is definitive: 90% of developers regularly use at least one AI tool at work for coding and development tasks. Not experimenting. Not dabbling. Using, daily, at their jobs.

Beyond raw adoption, 74% have moved past general-purpose chatbots like ChatGPT and adopted specialized AI developer tools — purpose-built coding assistants, editors, and agents designed for software engineering workflows.

Why are developers using AI?

- Developers perceive they are faster with AI
  - https://metr.org/blog/2025-07-10-early-2025-ai-experienced-os-dev-study/

- Developers ***may now actually develop faster*** with AI
  - The peer-reviewed evidence on this is mixed and complicated, many anecdotal reports from authoritative voices about improvements as of late 2025 onwards though
  - It speeds up code writing, but if the code is reviewed, tested, etc, the exact amount of "gain" (if any) isn't totally clear

- Companies are even mandating AI usage
  - https://medium.com/@mohamed-abdelmenem/metas-dashboard-tracks-your-ai-usage-daily-miss-80-your-job-s-at-risk-30f32d192146

Ryan Dahl the creator of NodeJS...



Ryan Dahl
@rough__sea

Following

This has been said a thousand times before, but allow me to add my own voice: the era of humans writing code is over. Disturbing for those of us who identify as SWEs, but no less true. That's not to say SWEs don't have work to do, but writing syntax directly is not it.

21:32 · 19 Jan 26 · **1.5M** Views

Something Flipped in December



Noah Mitchem Follow 10 min read · Feb 27, 2026

61 3 1

In July 2025, the most rigorous study ever conducted on AI coding tools found they made experienced developers 19% slower. In February 2026, Stripe is shipping over 1,000 AI-written pull requests per week into a codebase that processes over a trillion dollars annually.

That's not a typo. That's six months.

A good read: https://medium.com/@NMitchem/something-flipped-in-december-423e8b808262

AI is raising a number of concerns

- Quality of code
  - AI makes mistakes, what about security and accessibility?
    - e.g. https://cloudsecurityalliance.org/blog/2025/07/09/understanding-security-risks-in-ai-generated-code
- Developer burnout, skill erosion, and comprehension debt, verification debt
  - Developers may lose their understanding of the codebase when using AI to produce applications
    - Example: https://www.oreilly.com/radar/comprehension-debt-the-hidden-cost-of-ai-generated-code/
  - AI produced code should be verified, which has a cost
    - Article: https://www.kevinbrowne.ca/verification-debt-is-the-ai-eras-technical-debt/

AI is raising a number of concerns

- Does AI ***actually*** make developers faster?

- At least one good study shows in some cases developers perceived they were faster, but they were actually slower
  - Source: https://metr.org/blog/2025-07-10-early-2025-ai-experienced-os-dev-study/
  - This study has been highly cited, but even the authors have acknowledged it's just one study and may not generalize
  - This study is now very dated, AI has improved significantly since the study as well

Industry consensus isn't there yet

- AI and its impact on software development is basically "the" topic at the moment
  - But there truly isn't a real consensus yet
- Industry experts were gathered in the same Utah mountains the agile manifesto was produced to discuss the topic
  - Almost as if they had similar industry shaping ambitions as the agile manifesto
- This paper on the Future of Software Engineering was produced

Future of Software Engineering

"We kept asking the same question in every room: if AI handles the code, where does the engineering actually go? Nobody had the same answer.
But everybody agreed the question is urgent."

"The retreat didn't produce a roadmap. It produced a shared understanding that the map is being redrawn and that the people best positioned to draw it are the ones willing to admit how much they don't yet know."

Future of Software Engineering

- Industry heavyweights like Martin Fowler attended this, and that's what they got out of it!

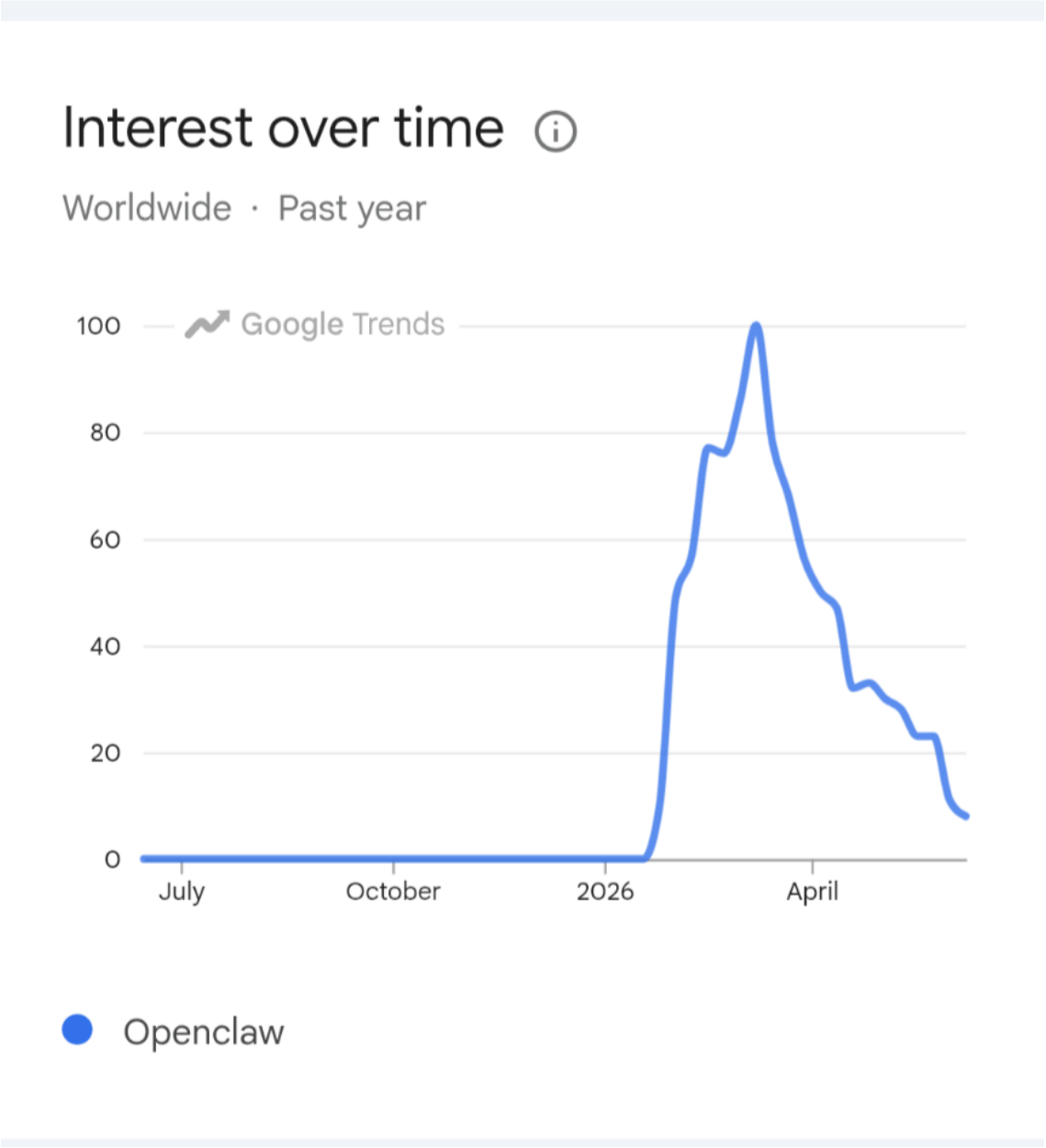
- A few takeaways from this…

- Don't feel bad if you don't know how to handle this new era of software development with AI
  - If you admit you "don't know for sure", that's a strength right now, not a lack of knowledge

I was told by a student in February 2026 that if I didn't teach "multiagent software development workflows with OpenClaw" (released November 2025) that I was "not teaching modern software development" (exact words).

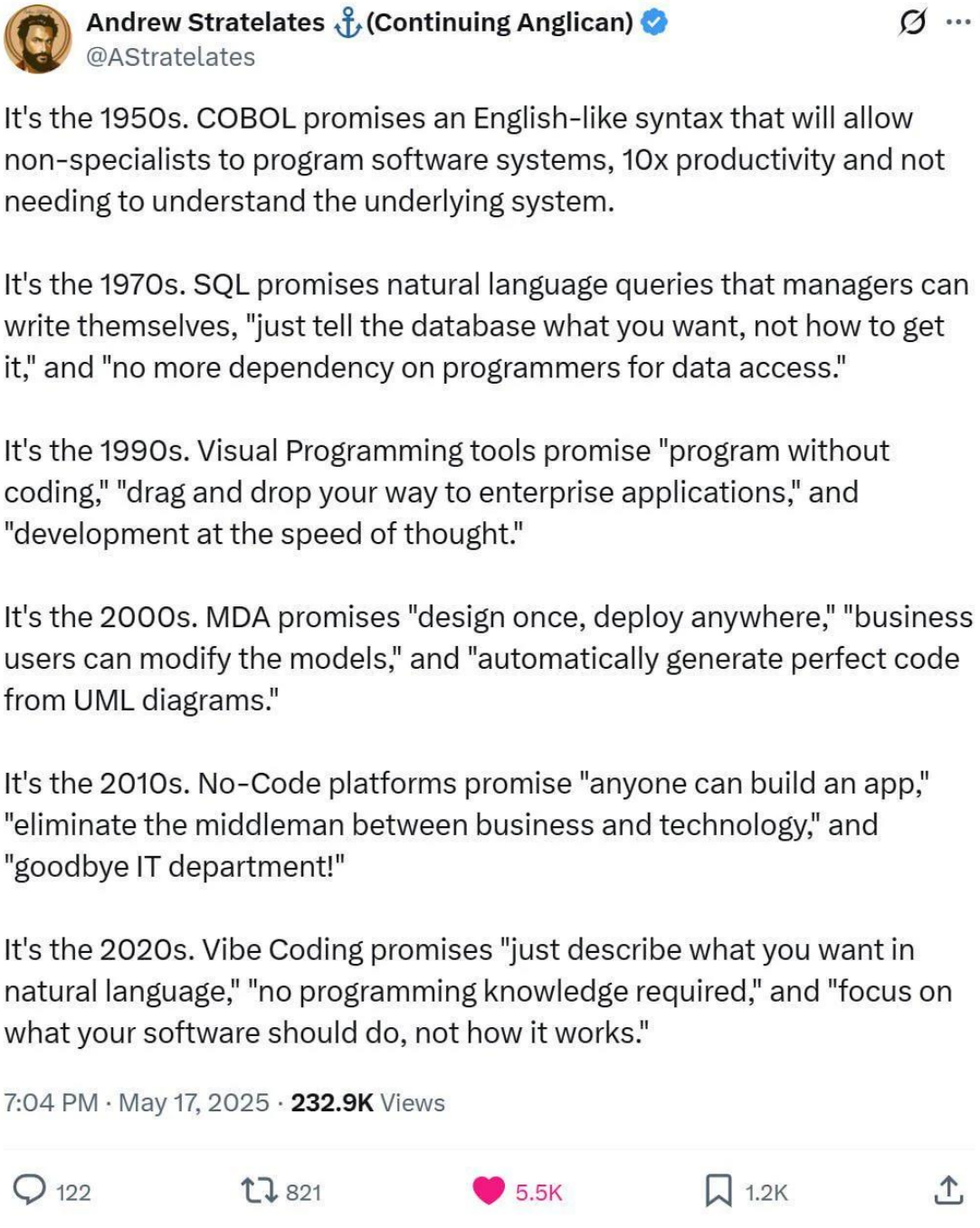
Well, let's see how OpenClaw is doing lately (see right).

This is NOT a criticism of the student's point, and the enthusiasm and interest is ***welcome***. But let's also recognize this is a space that's very rapidly changing!



There is also a long history of overhype for certain technologies to eliminate or fundamentally alter coding.

It's notable that none of these, except maybe high-level languages like COBOL, reached 90% adoption rates within such a short time. High-level languages may have been overhyped, but they did deliver significant gains.



Andrew Stratelates ⚓(Continuing Anglican)
@AStratelates

It's the 1950s. COBOL promises an English-like syntax that will allow non-specialists to program software systems, 10x productivity and not needing to understand the underlying system.

It's the 1970s. SQL promises natural language queries that managers can write themselves, "just tell the database what you want, not how to get it," and "no more dependency on programmers for data access."

It's the 1990s. Visual Programming tools promise "program without coding," "drag and drop your way to enterprise applications," and "development at the speed of thought."

It's the 2000s. MDA promises "design once, deploy anywhere," "business users can modify the models," and "automatically generate perfect code from UML diagrams."

It's the 2010s. No-Code platforms promise "anyone can build an app," "eliminate the middleman between business and technology," and "goodbye IT department!"

It's the 2020s. Vibe Coding promises "just describe what you want in natural language," "no programming knowledge required," and "focus on what your software should do, not how it works."

7:04 PM · May 17, 2025 · **232.9K** Views

122 821 5.5K 1.2K

Future of Software Engineering

- Our industry goes through periods of rapid change like this sometimes where ***experimentation*** is normal
  - e.g. experiments with different software development approaches in the 1990s led to Agile in the 2000s decade

- In the absence of clear best practices, try to have fun experimenting and learning in phases like this
  - Opportunities like this are kind of "generational" in occurrence, and they give us a chance to innovate and be the inventor!
  - But keep an open mind, rapid change goes in both directions

- The paper discusses how developers may experience a loss of identity, this more emotional reaction is normal too

Future of Software Engineering

- If AI can generate the code, where does the more rigorous engineering work go?
  - i.e. how do we know we've built the right thing the right way, if AI is taking on so much of the building?
- Increased importance of specifications, unit tests, and other artifacts such as constraints
  - This is why we need to talk about this topic in this course
- Are specifications replacing code?
  - In some sense… when code is generated from specifications, reviewing specifications and optimizing them becomes more important

Future of Software Engineering

- AI generates code non-deterministically, the same prompt can produce different code
- Tests can be written by AI, but AI can "cheat" by writing tests that align to its own output
- Human-written acceptance criteria or tests can offer a rigorous, well-defined deterministic target for AI generated code to satisfy
  - If the AI builds tests based on acceptance criteria and the human reviews them, this can work too!

Recent history

- ChatGPT was released in November 2022
  - Led to massive surge in interest in AI for software development, investment in AI in general, etc.
- GitHub Copilot released outside of technical preview in June 2022
  - Allowed developers to code with AI assistance
  - Codex model descended from GPT3 by OpenAI was initially used, then much improved GPT4, and later on models could be chosen

Early AI in software development

- Initially AI used as part of GitHub Copilot and similar application was largely used as a form of inline autocompletion
  - Copilot docs refer to these as ***completions***

- e.g. if you wrote a function header, VS Code and GitHub Copilot would fill in the rest of the function body for you
  - You could just hit tab to complete it

- You could use AI for other purposes, like generating fake data, etc, but essentially we can describe this early stage as "AI for advanced auto-completion"

Conversational prompt inputs

- Around 2023-25 there was a shift to a more conversational prompt style of interaction
- e.g. Copilot chat released in 2023, Claude Code released in 2025, Cursor released in 2023
- These tools are different, but they all placed a greater emphasis on writing prompts to create code rather than auto-completion
  - i.e. developer may write a prompt asking for a feature to be implemented in code, AI attempts to write the code
  - Brings the work up a layer of abstraction, don't even need to necessarily look at the code anymore while developing (though this may obviously have serious negative consequences)



Andrej Karpathy
@karpathy

There's a new kind of coding I call "vibe coding", where you fully give in to the vibes, embrace exponentials, and forget that the code even exists. It's possible because the LLMs (e.g. Cursor Composer w Sonnet) are getting too good. Also I just talk to Composer with SuperWhisper so I barely even touch the keyboard. I ask for the dumbest things like "decrease the padding on the sidebar by half" because I'm too lazy to find it. I "Accept All" always, I don't read the diffs anymore. When I get error messages I just copy paste them in with no comment, usually that fixes it. The code grows beyond my usual comprehension, I'd have to really read through it for a while. Sometimes the LLMs can't fix a bug so I just work around it or ask for random changes until it goes away. It's not too bad for throwaway weekend projects, but still quite amusing. I'm building a project or webapp, but it's not really coding - I just see stuff, say stuff, run stuff, and copy paste stuff, and it mostly works.

3:17 PM · Feb 2, 2025 · **6.8M** Views

Vibe coding

- The term vibe coding was coined by Canadian computer scientist and co-founder of OpenAI Andrej Karpathy

- **Vibe coding** refers to a software development workflow where developers describe projects and features in natural language and allow the AI to do the implementation work (coding, testing, deploying)
  - The precise definition is still debatable, e.g. is it still vibe coding if you sometimes look at the code and modify?
  - But this does bring the work up a layer of abstraction

Vibe coding

- Vibe coding has been criticized for good reasons
  - The "prompt and hope" workflow lacks rigor, and this lack of rigor leads to problems

- AI can make mistakes ("hallucinate") and critical errors can result
  - Bugs in code, including security holes
  - Leaked passwords, API keys, etc.
  - If you're not looking at the code, what's stopping these vulnerabilities from reaching production?

Vibe coding

- Cognitive and verification debt increase as developers lose their understanding of the underlying code
  - Who is accountable when something breaks?
- AI produced code can be verbose, inelegant
  - Where does refactoring occur to ensure the code is clean and well-architected?
  - How well will this approach scale to larger code bases, and longer term?
- Many professional software developers heavily criticize vibe coding for these and other reasons

Vibe coding

- The flip side is that there are success stories
  - e.g. entirely vibe coded: https://www.rushhome.com/
  - Some of these success stories are oversold, yes
  - Apps and websites have been released before AI successfully which did not require coding
    - e.g. WordPress websites built with themes + plugins

- Vibe coding can also be excellent for prototyping
  - We don't really need to worry about security concerns or code quality issues with prototypes
  - It's a way for even non-technical people to get a high-fidelity interactive prototype up and running

Prompt engineering

- **Prompt engineering** is the art and science of designing and optimizing instructions to guide AI models, such as ChatGPT

- Prompts can be improved by providing clear and unambiguous instructions, providing more relevant context, and providing more constraints (telling the model what not to do)
  - Much of what we learned about writing better requirements applies to writing better prompts!



Andrej Karpathy
@karpathy

A lot of people quote tweeted this as 1 year anniversary of vibe coding. Some retrospective -

I've had a Twitter account for 17 years now (omg) and I still can't predict my tweet engagement basically at all. This was a shower of thoughts throwaway tweet that I just fired off without thinking but somehow it minted a fitting name at the right moment for something that a lot of people were feeling at the same time, so here we are: vibe coding is now mentioned on my Wikipedia as a major memetic "contribution" and even its article is longer. lol

The one thing I'd add is that at the time, LLM capability was low enough that you'd mostly use vibe coding for fun throwaway projects, demos and explorations. It was good fun and it almost worked. Today (1 year later), programming via LLM agents is increasingly becoming a default workflow for professionals, except with more oversight and scrutiny. The goal is to claim the leverage from the use of agents but without any compromise on the quality of the software. Many people have tried to come up with a better name for this to differentiate it from vibe coding, personally my current favorite "agentic engineering":

- "agentic" because the new default is that you are not writing the code directly 99% of the time, you are orchestrating agents who do and acting as oversight.
- "engineering" to emphasize that there is an art & science and expertise to it. It's something you can learn and become better at, with its own depth of a different kind.

Agentic engineering

- **Agentic engineering** is a currently emerging term with similar and overlapping definitions…
  - Agentic engineering is the practice of designing systems where AI agents can plan tasks, use tools, make decisions in steps, and complete outcomes with human supervision. [source]
  - Agentic engineering — the discipline of designing systems where AI agents plan, write, test, and ship code under structured human oversight. [source]

- The word "agentic" refers to the tool, the word "engineering" refers to treating this as a practice which can be learned, improved, done better and more rigorously and intentionally

What are these AI agents?

- The term **AI agent** is ***also*** emerging and a standard definition does not exist!
  - What a fun time to be a teacher or student…
- But essentially an AI agent is AI which can autonomously perform multi-step tasks to achieve a goal
- Agentic AI has been key to enabling ***both*** vibe coding and agentic engineering
- i.e. the ***tool*** is agentic AI, the ***process*** is what separates vibe coding from agentic engineering
  - Vibe coding and agentic engineering exist along a spectrum going from less structure to more structure

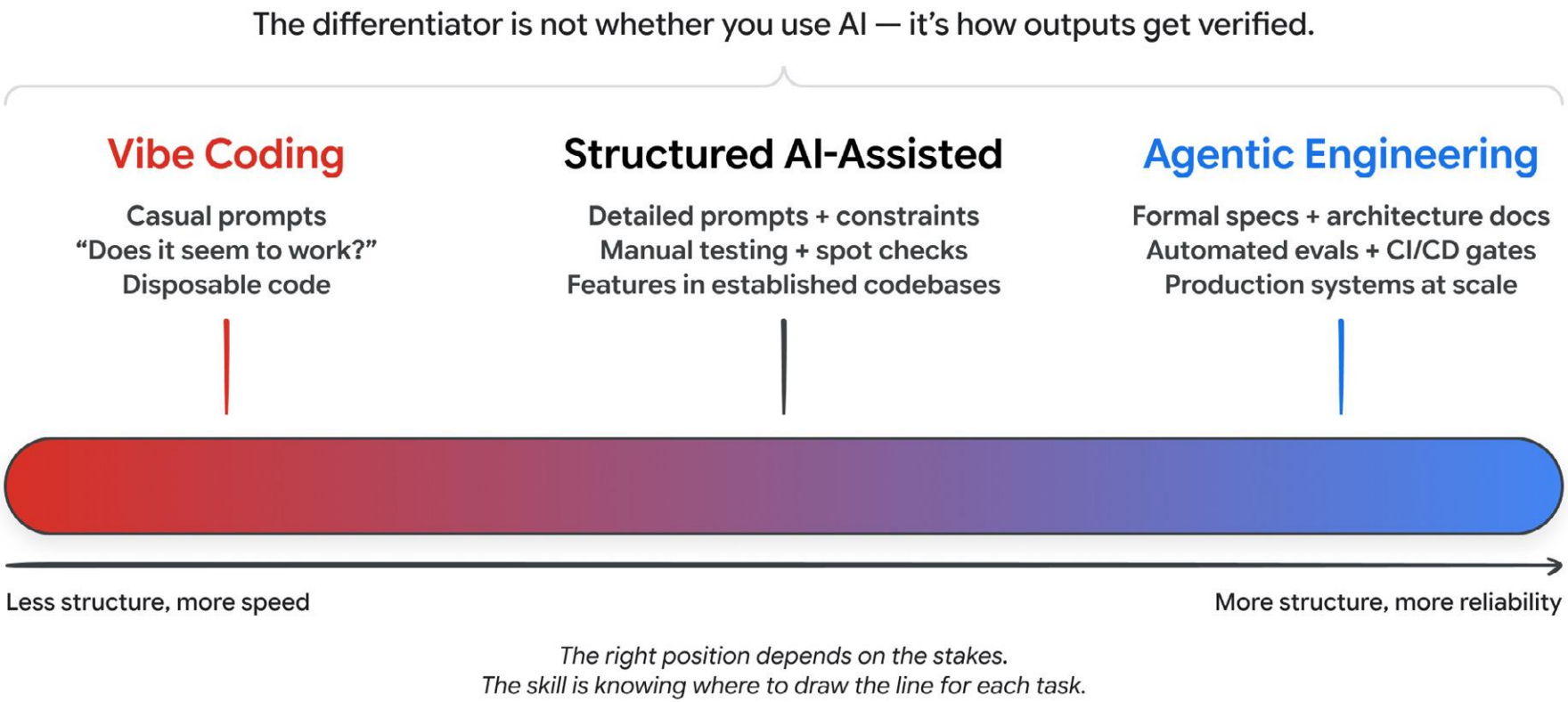
Vibe Coding vs. Agentic Coding

Comparison Table

Aspect	Vibe Coding	Agentic Coding
Approach	Free-flowing, creative	Structured, goal-oriented
Process	Trial and error,experimentation	Step-by-step, planned approach
Flexibility	Highly flexible, fast prototyping	More rigid, well-defined
Type of Projects	Rapid development, idea generation	Larger projects, complex systems
Focus	Intuitive, user-driven	Scalable solutions, business-driven results
Tools & Techniques	Uses basic frameworks, often limited to prototypes	Uses advanced frameworks and tools for stability
Time to Complete	Shorter, quick results	Longer, requires thorough testing and iteration
Ideal Use Case	Quick apps, MVPs, early-stage projects	Enterprise-level solutions, full-scale development

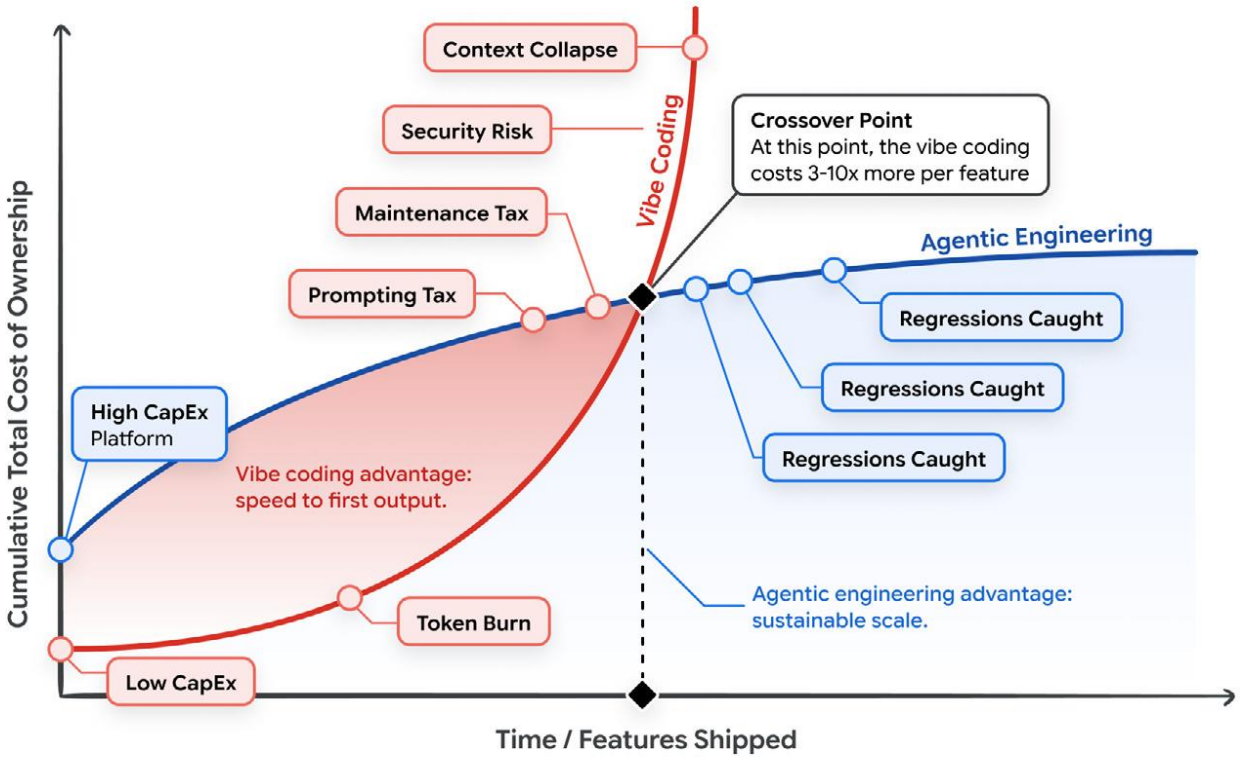
Source: https://www.sourcedesk.io/blog/vibe-coding-vs-agentic-coding-what-beginners-need-to-know-about-ai-driven-software-development-in-2026

Spectrum



Source: https://addyosmani.com/blog/new-sdlc-vibe-coding/

Economic comparison



Vibe Coding

12%

CapEx
Min Investment

OpEx
High Running Costs

- Rapid prototyping, slow scaling
- High friction for long-term maintenance
- Economic dead-end for complex systems

Agentic Engineering

12%

CapEx
Upfront Platform Design

OpEx
Low Marginal Running Costs

- Controlled iteration, fast scaling
- Low friction for automatic updates
- Economically sustainable for mature codebases

AI agents

- In the context of tools like GitHub Copilot CLI and Claude Code, when given a prompt to implement a feature these tools can do things like...
  - Edit multiple code files
  - Run terminal commands
    - e.g. to test code, commit code, deploy code
  - Sequence these actions in the right combinations to implement the feature

- This is the tool aspect, the "agentic" part, of "agentic engineering"
  - And note that it's agents, plural, i.e. multiple agents working in parallel may do these things

Agentic engineering

- Prompting and prompt engineering doesn't go away in agentic engineering
  - Prompting is still a key form of input into the process
- Agentic engineering effectively includes prompt engineering as a practice
- Agentic engineering adds some kind of process designed to improve the output
  - That "process" isn't one universal thing, agentic engineering is referring to having some actual process
  - Like how "software development processes" refer to waterfall, agile and other processes

Agentic engineering

- At a basic level, agentic engineering could be achieved by writing prompts to implement features where we specify:
  - Well-defined requirements for a feature
  - Constraints (non-negotiable boundaries)
    - i.e. what not to do, what not to change
    - i.e. practices to follow
  - Acceptance criteria

- And then reviewing the code and tests that are produced by the prompt

- This could be done in a "loop" to build an application and its features over time
  - But agentic engineering could also involve more complex processes, tooling, documentation, etc.

Agentic engineering

- Agentic engineering may involve more than one agent
- It might include multiple agents working in parallel on different features
- It might include multiple agents working at different parts in a pipeline (specs -> dev -> test)
  - One for implementation, one for UX, one for testing, etc.

Agentic engineering

- There is not one "right" way to do agentic engineering, but there are emerging best practices and ideas

- But the great news is that all of our requirements and specifications skills carry over ***almost directly*** to agentic engineering

- We could say that agentic engineering is development done at the level of requirements and specifications

Spec-Driven Development (SDD)

- **Spec-Driven Development (SDD)** is a development approach where specifications, not code, are the primary artifact

- Instead of prompting to implement a feature, the feature is defined in a spec

- The AI takes on a role a bit like a compiler turning high-level languages into machine code, but here it turns specifications into code, tests, etc.

Spec-Driven Development (SDD)

- **Spec-first development** is when a well-defined specification is created before any code is generated

- **Spec-anchored development** is when the specification is evolved alongside the code
  - As new features are added or features are modified, the spec is updated
  - This helps to "anchor" the AI making changes such that changes do not contravene the spec, i.e. a new feature's implementation breaks an old feature

Spec-Driven Development

- **Spec-as-source development** is when the developer does not manually touch the code and only the spec is altered
  - The code may be automatically updated when the spec changes as part of this

- The concepts of SDD and agentic engineering overlap, in that much agentic engineering uses SDD in some form or another
  - It may not be identified as such by name though

SDD vs Agentic Engineering

- SDD is the ***what*** and ***how***
  - The blueprint defining intent, specifications constraints, and verification
- Agentic Engineering is the ***who*** and ***where***
  - Orchestrating AI agents to autonomously build and test that blueprint, possibly in parallel
- Most agentic engineering is done primarily in a spec-anchored way
  - In that there is at least some specification files in an application's repository to provide context to the AI

SDD vs Agentic Engineering

- But often development begins technically spec-first

- And if specs are rigorous enough, it should ***technically*** be possible to generate the app entirely from the specs
  - Though the application wouldn't very likely be identical to previous version due to non-deterministic nature of AI output

- Exactly how the specifications are represented and to what degree can vary
  - But basically, in-between prompting with the AI, specs can be maintained to give the AI context, in a persistent format like markdown files

Specs in agentic engineering

- It could be a single markdown file which provides a big picture on the project's purpose and main points
- It could be multiple markdown files that specify different features, capabilities, aspects, etc.
- Or it could be a large markdown file effectively acting as a software requirements specification (SRS) document targeting AI as the reader

Context

- In agentic engineering **context** is the entire pool of information available to the AI agent
  - Specs are a form of context which can help the agent

- The code is part of the context, and an AI agent can often use the existing code very well to frame how to make decisions
  - i.e. implicit details in code act as a good frame for the AI agent, it doesn't ***need*** explicit spec files to make changes successfully

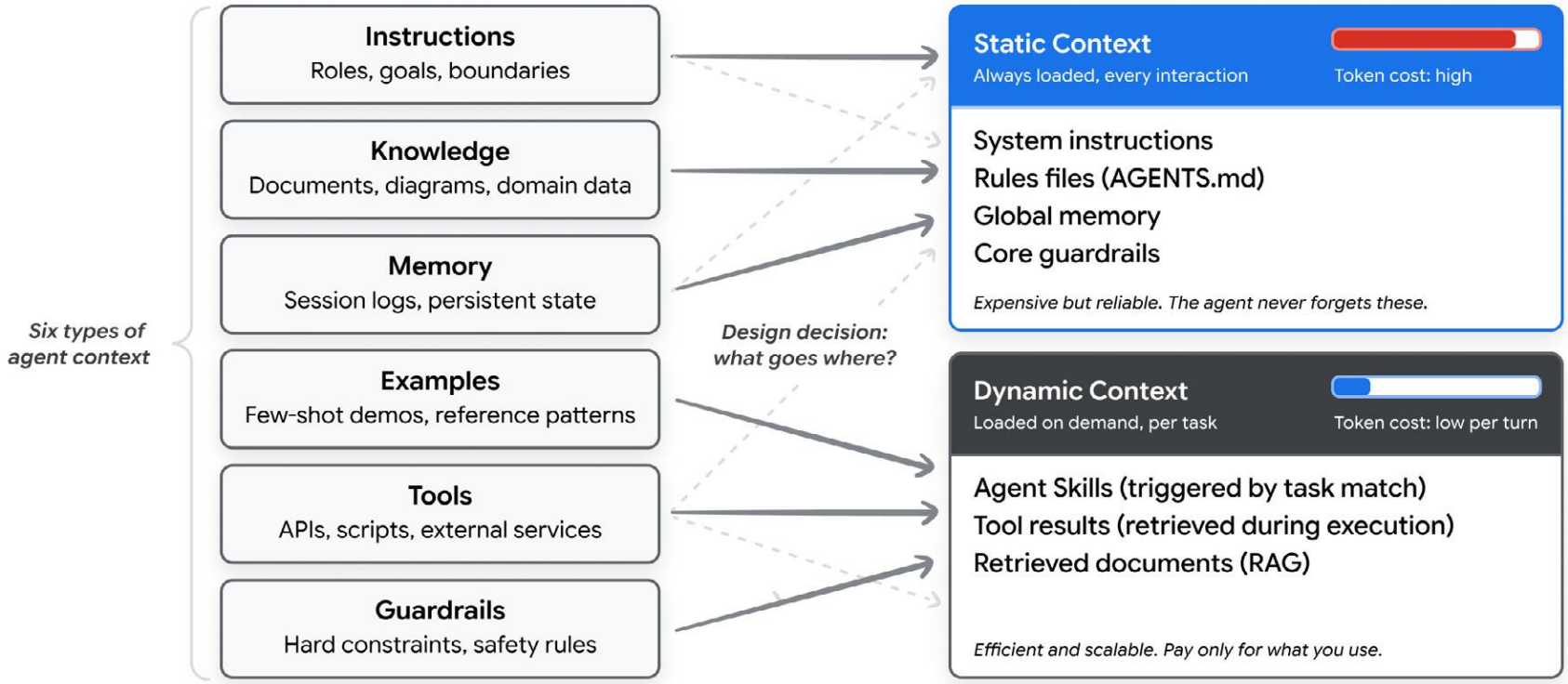
Context

- Context can be static like files with specs in them, but it can also be dynamically pulled from online sources (documents, APIs, etc.)

- There is an important trade-off between more or less context too
  - More context positives: better decisions, less incorrect implementation of requirements
  - More context negatives: higher cost, higher latency, attention dilution (more for the AI to focus on)

- Text is broken down into tokens for AI, cost is per token, so more context == more cost

Context



The best systems treat this as a first-class architectural decision, reviewed and versioned like code.

Source: https://addyosmani.com/blog/new-sdlc-vibe-coding/

Claude Code

- Claude Code is one of the most popular agentic coding tools
- It works by just prompting from the terminal program, though "commands" are available via /
- Claude Code projects can have a CLAUDE.md file which serves as a persistent, project-specific instruction manual for Claude Code
  - Can create a "global" CLAUDE.md file too
  - AGENTDS.md is the more universally standard equivalent

CLAUDE.md

- Can be created with /init command
- Project problem, scope, tech stack, architecture can be defined in CLAUDE.md
- Also workflow and behaviour as to how Claude Code itself should behave when working on your project
- To an extent, this is a spec in agentic engineering, though it tends to be high-level and meta in use
  - It also tends to be kept on the shorter side

Product Requirements Document

- A Product Requirements Document (PRD) is a formal document that defines what a software product (or feature) should do
  - But not precisely how everything needs to be implemented
  - A PRD can overlap with an Software Requirements Specification (SRS) conceptually, but it's generally up a layer of abstraction, focused more on why
- It aligns stakeholders, developers, designers, and testers around a shared understanding of the problem being solved and the outcomes expected.
- It serves as the contract between the people who define the product and the people who build it.

Product Requirements Document

- A PRD can be used to provide requirements context to the agent
- One agentic engineering technique is to create a prd.md file which becomes the spec for the project
- If this file became large enough, it could be broken down into multiple files covering different features, aspects, etc.

A PRD might include…

- Problem statement
- Scope, goals
- Requirements
  - Functional requirements
  - Non-functional requirements
  - Other categories
- Constraints (non-negotiable boundaries)
- Acceptance criteria
  - What does success look like?

A PRD might include...

- Overall architecture specification
- Error handling
  - Fallback behaviours
  - Failure modes
- User personas or target user summary
- Context sources
  - What additional sources should be used for context
- Open questions
  - What are the unresolved questions?
  - It's important to keep ambiguity visible so it is resolved deliberately instead of assumed by the AI

A PRD might include…

- Dependencies
  - What does the software (and agent) rely on in terms of other software, APIs, etc.
- Human-in-the-Loop authority
  - At what point should the AI pause for human input when carrying out tasks?
- Agent authority
  - What decisions should the AI agent make autonomously?
- Assumptions
  - What assumptions are being made as part of developing the application

Using a PRD in agentic engineering

- We could define a PRD manually or review one created with AI assistance and modify it
- We could then make our changes by modifying the PRD, and telling the agent to look at it
  - Or we could build that into the workflow of the agent
  - e.g. by adding some text in CLAUDE.md in the case of Claude Code
  - We can ***still*** modify the code directly or with agents/prompts, and then update the PRD afterwards
- We can remind Claude Code to ask us clarifying questions as part of these prompts and/or files

Using a PRD in agentic engineering

- We get the benefit of a "master" copy of the specifications and other details
  - More rigorously defined requirements
  - As we add new features, older ones should be respected
- Important to own the acceptance criteria as a human even if the AI helps to write them
  - If an AI writes both the code and tests, its easy for it to make code to pass them
  - But when AI creates test from **our** acceptance criteria it will try to use those as the basis for test creation

Agentic engineering

- Using a PRD is just one technique, again, we could break this concept into many files with different roles, purposes and content
  - The technique actually is adopted by many real projects too
  - A PRD maps pretty nicely to those who have an understanding of software requirements and specifications
- Agentic engineering has so many more issues, concerns, techniques, etc.

Agentic engineering

- Agentic tools fail confidently and quietly
  - When they make a mistake, you may not realize it
  - This is in contrast to code not compiling or an application crashing
- Prompt injections are a security threat
  - If your AI agent is reading sources online, a prompt could be included in them which hacks your system!
- There's an art to agentic engineering
  - Human vs agent decision making, the level of context to provide (and what context and how)

Agentic engineering

- **Model Context Protocol (MCP)** is an open-source standard created by Anthropic that allows AI models to securely connect to external data sources and tools
  - Some claim these have become less important as AI agents have become more capable
- **Skills** are modular, reusable instruction packages that teach your AI coding agent specialized domain knowledge and workflows
  - Like MCP, the importance and usage of skills in in flux